



torte: Reproducible Feature-Model Experiments à la Carte

ICSE 2026 — April 12–18 — Rio de Janeiro, Brazil

Elias Kuiter¹

University of Magdeburg¹, Germany

Implementing Configurable Software Systems

A Configurable Graph

```
class Node {  
    #ifdef LABELED  
        std::string label;  
    #endif  
    #ifdef COLORED  
        std::string color;  
    #endif  
};  
  
class Edge {  
    #ifdef DIRECTED  
        Node from, to;  
    #elif UNDIRECTED && HYPER  
        std::set<Node> nodes;  
    #endif  
};
```

Product Line Implementation

(here: C++ with C preprocessor)

Implementing Configurable Software Systems

A Configurable Graph

```
class Node {  
    #ifdef LABELED  
        std::string label;  
    #endif  
    #ifdef COLORED  
        std::string color;  
    #endif  
};  
  
class Edge {  
    #ifdef DIRECTED  
        Node from, to;  
    #elif UNDIRECTED && HYPER  
        std::set<Node> nodes;  
    #endif  
};
```

Product Line Implementation
(here: C++ with C preprocessor)

~~#define~~
LABELED
DIRECTED

Labeled Directed

```
class Node {  
    std::string label;  
};  
  
class Edge {  
    Node from, to;  
};
```

Configuration

Product Implementation

Implementing Configurable Software Systems

A Configurable Graph

```
class Node {  
    #ifdef LABELED  
        std::string label;  
    #endif  
    #ifdef COLORED  
        std::string color;  
    #endif  
};  
  
class Edge {  
    #ifdef DIRECTED  
        Node from, to;  
    #elif UNDIRECTED && HYPER  
        std::set<Node> nodes;  
    #endif  
};
```

Product Line Implementation
(here: C++ with C preprocessor)

*#define
LABELED
DIRECTED*

*#define
COLORED
UNDIRECTED
HYPER*

Configuration

Labeled Directed

```
class Node {  
    std::string label;  
};  
  
class Edge {  
    Node from, to;  
};
```

Colored Undirected Hyper

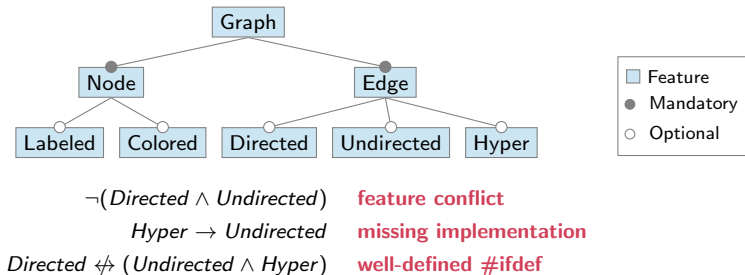
```
class Node {  
    std::string color;  
};  
  
class Edge {  
    std::set<Node> nodes;  
};
```

Product Implementation

Modeling Features and their Dependencies

Feature Models

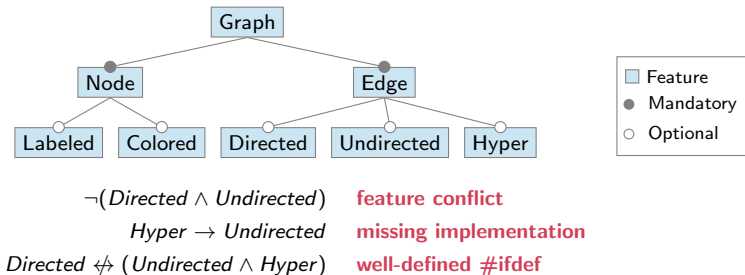
- tree models **features**
- cross-tree **constraints** model dependencies
- solver-based **analyses** can be used to understand the configuration space better



Modeling Features and their Dependencies

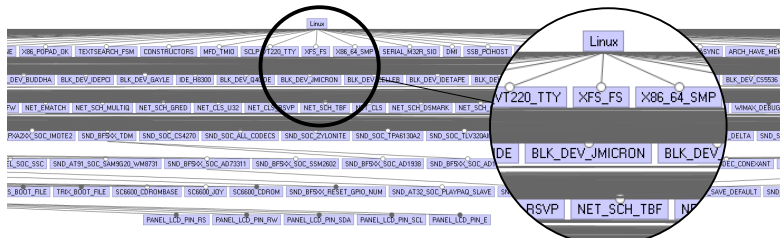
Feature Models

- tree models **features**
- cross-tree **constraints** model dependencies
- solver-based **analyses** can be used to understand the configuration space better



The Linux Kernel

- ≈ 20000 features [2025]
- $> 10^{1600}$ products [2025]
- 114 dead features [2013]
- 151 reverse dependency bugs [2019]

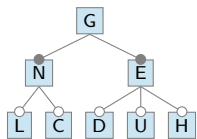


Feature-Model Analysis – Background

Feature-Model Analysis – Background

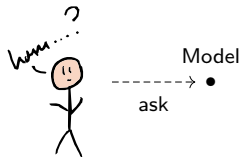


Feature Model



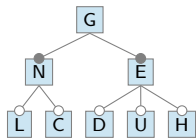
$\neg(D \wedge U)$
 $H \rightarrow U$
 $D \not\leftrightarrow (U \wedge H)$

Feature-Model Analysis – Background



----->
interactive

Feature Model

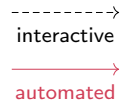
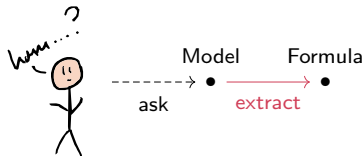


$\neg(D \wedge U)$
 $H \rightarrow U$
 $D \nleftrightarrow (U \wedge H)$

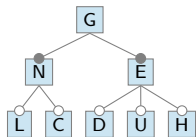
Core Feature $F?$

#Configurations?

Feature-Model Analysis – Background



Feature Model



$\neg(D \wedge U)$
 $H \rightarrow U$
 $D \nleftrightarrow (U \wedge H)$

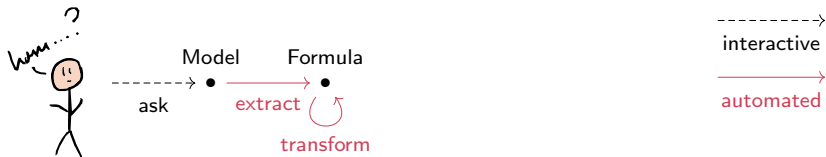
Formula

G
 $\wedge (N \leftrightarrow G) \wedge (E \leftrightarrow G)$
 $\wedge ((L \vee C) \rightarrow N)$
 $\wedge ((D \vee U \vee H) \rightarrow E)$
 $\wedge \neg(D \wedge U) \wedge (H \rightarrow U)$
 $\wedge (D \nleftrightarrow (U \wedge H))$

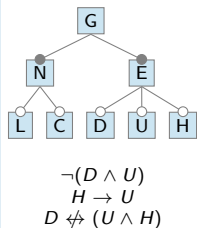
Core Feature F ?

#Configurations?

Feature-Model Analysis – Background



Feature Model



Formula

G
 $\wedge (N \leftrightarrow G) \wedge (E \leftrightarrow G)$
 $\wedge ((L \vee C) \rightarrow N)$
 $\wedge ((D \vee U \vee H) \rightarrow E)$
 $\wedge \neg(D \wedge U) \wedge (H \rightarrow U)$
 $\wedge (D \nleftrightarrow (U \wedge H))$

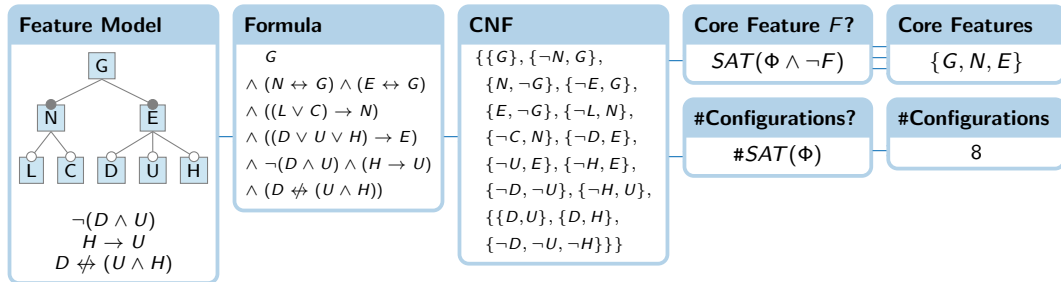
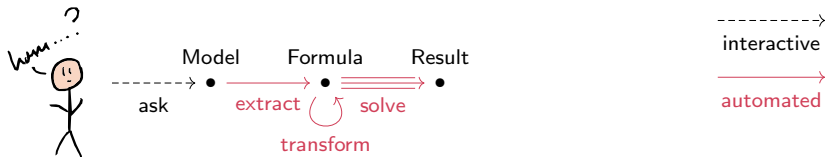
CNF

$\{\{G\}, \{\neg N, G\},$
 $\{N, \neg G\}, \{\neg E, G\},$
 $\{E, \neg G\}, \{\neg L, N\},$
 $\{\neg C, N\}, \{\neg D, E\},$
 $\{\neg U, E\}, \{\neg H, E\},$
 $\{\neg D, \neg U\}, \{\neg H, U\},$
 $\{\{D, U\}, \{D, H\},$
 $\{\neg D, \neg U, \neg H\}\}$

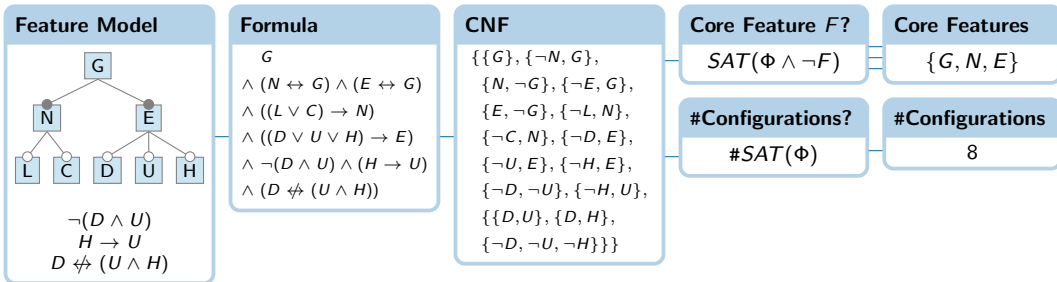
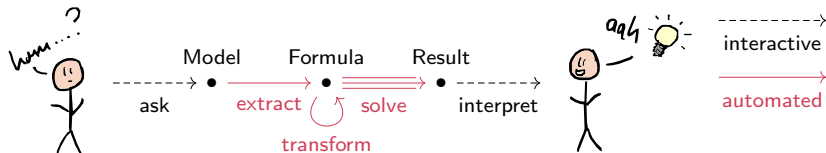
Core Feature F ?

#Configurations?

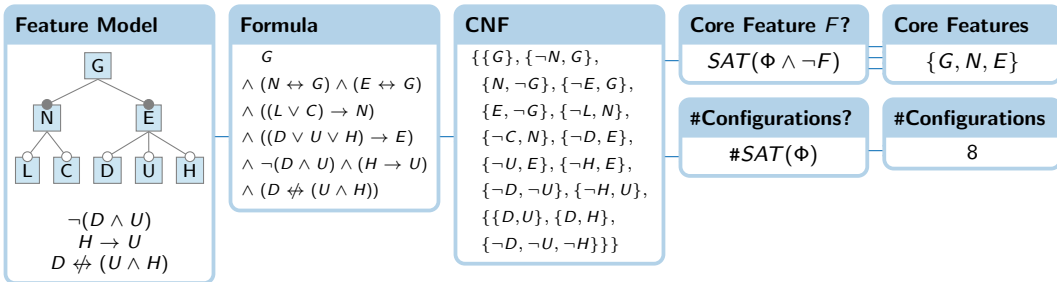
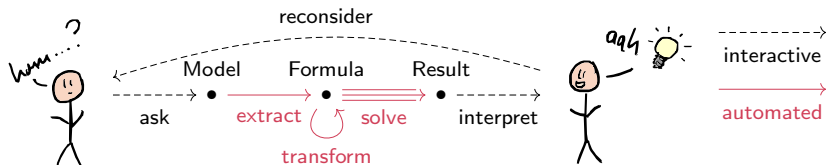
Feature-Model Analysis – Background



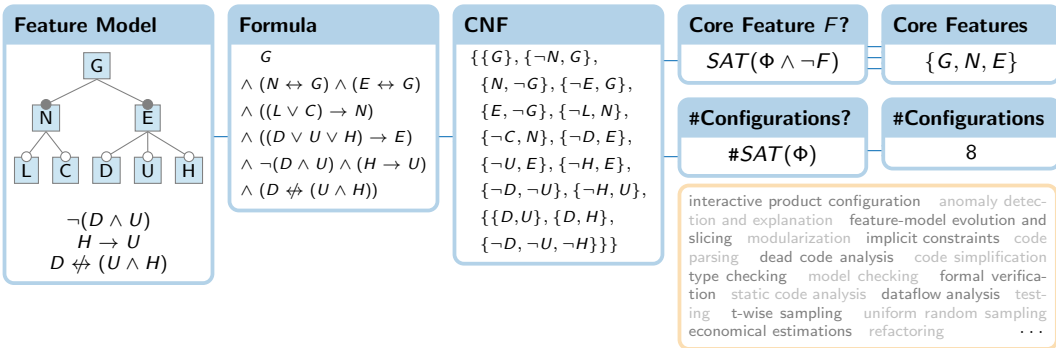
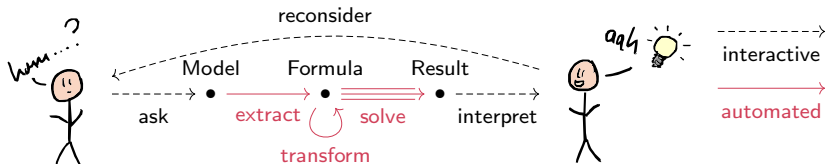
Feature-Model Analysis – Background



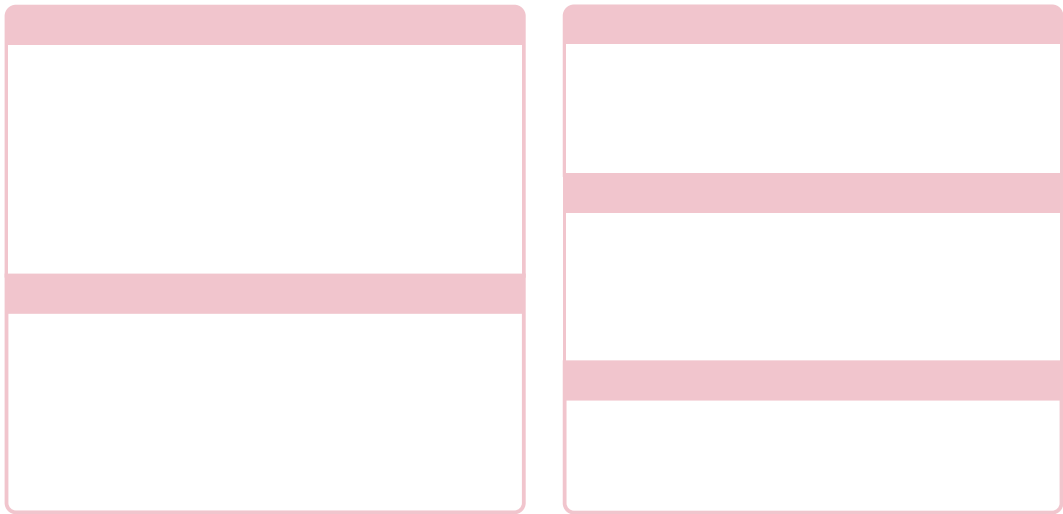
Feature-Model Analysis – Background



Feature-Model Analysis – Background



Feature-Model Analysis – Open Challenges



Feature-Model Analysis – Open Challenges

History Extraction

BusyBox (20 years) axTLS (14 years) Buildroot (17 years) EmbToolkit (6 years) Freetz-NG (19 years) L4Re (9 years) toybox (19 years) uClibc (10 years) **Linux (24 years)** uClibc-ng (10 years) coreboot u-boot Crosstool-NG Soletta FreeWRT QEMU RIOT Unikraft Zephyr esp-idf sdk-nrf ...

KConfig **evolves/diverges** $\Rightarrow$ no consistent histories

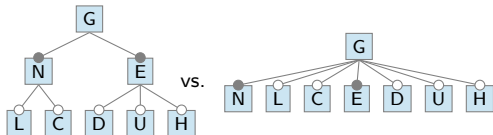
Feature-Model Analysis – Open Challenges

History Extraction

BusyBox (20 years) axTLS (14 years) Buildroot (17 years)
EmbToolkit (6 years) Freetz-NG (19 years)
L4Re (9 years) toybox (19 years) uClibc (10 years)
Linux (24 years) uClibc-ng (10 years) coreboot u-boot
Crosstool-NG Soletta FreeWRT QEMU RIOT
Unikraft Zephyr esp-idf sdk-nrf ...

KConfig **evolves/diverges** $\Rightarrow$ no consistent histories

Hierarchy Extraction



extracted models are **flat** $\Rightarrow$ loses developer intent

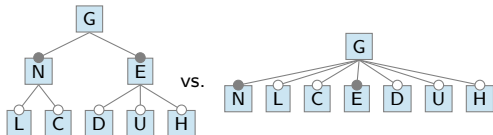
Feature-Model Analysis – Open Challenges

History Extraction

BusyBox (20 years) axTLS (14 years) Buildroot (17 years) EmbToolkit (6 years) Freetz-NG (19 years) L4Re (9 years) toybox (19 years) uClibc (10 years) **Linux (24 years)** uClibc-ng (10 years) coreboot u-boot Crosstool-NG Soletta FreeWRT QEMU RIOT Unikraft Zephyr esp-idf sdk-nrf ...

KConfig **evolves/diverges** $\Rightarrow$ no consistent histories

Hierarchy Extraction



extracted models are **flat** $\Rightarrow$ loses developer intent

Automation and Reproducibility

913bf31 · 10 years ago outdated dependencies

linked, for GNU/Linux 2.0.0, precompiled binaries

experiments are **rarely reproduced**

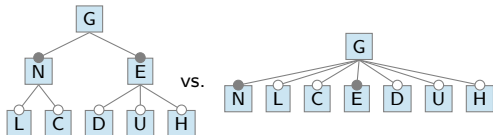
Feature-Model Analysis – Open Challenges

History Extraction

BusyBox (20 years) axTLS (14 years) Buildroot (17 years) EmbToolkit (6 years) Freetz-NG (19 years) L4Re (9 years) toybox (19 years) uClibc (10 years) **Linux (24 years)** uClibc-ng (10 years) coreboot u-boot Crosstool-NG Soletta FreeWRT QEMU RIOT Unikraft Zephyr esp-idf sdk-nrf ...

KConfig **evolves/diverges** $\Rightarrow$ no consistent histories

Hierarchy Extraction



extracted models are **flat** $\Rightarrow$ loses developer intent

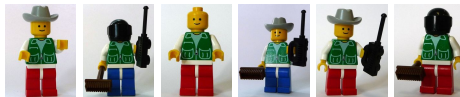
Automation and Reproducibility

913bf31 · 10 years ago outdated dependencies

linked, for GNU/Linux 2.0.0, precompiled binaries

experiments are **rarely reproduced**

Reusability and Interoperability



handcrafted experiments, limited **comparability**

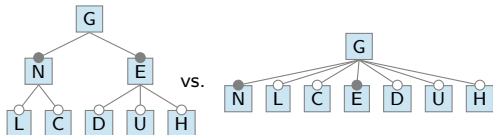
Feature-Model Analysis – Open Challenges

History Extraction

BusyBox (20 years) axTLS (14 years) Buildroot (17 years) EmbToolkit (6 years) Freetz-NG (19 years) L4Re (9 years) toybox (19 years) uClibc (10 years) **Linux (24 years)** uClibc-ng (10 years) coreboot u-boot Crosstool-NG Soletta FreeWRT QEMU RIOT Unikraft Zephyr esp-idf sdk-nrf ...

KConfig **evolves/diverges** $\Rightarrow$ no consistent histories

Hierarchy Extraction



extracted models are **flat** $\Rightarrow$ loses developer intent

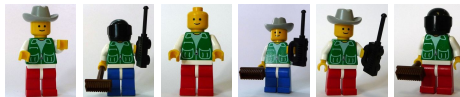
Automation and Reproducibility

913bf31 · 10 years ago outdated dependencies

linked, for GNU/Linux 2.0.0, precompiled binaries

experiments are **rarely reproduced**

Reusability and Interoperability



handcrafted experiments, limited **comparability**

Ease of Use

C(++) Scala Rust Java Bash Python ...

complex orchestration of heterogeneous tools



torte – An Experimentation Platform For Feature-Model Analysis

torte – An Experimentation Platform For Feature-Model Analysis

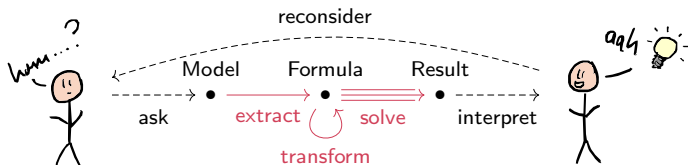
torte 🍰

Roll your own experiments!



 /ekuiter/ 🍰

torte – An Experimentation Platform For Feature-Model Analysis



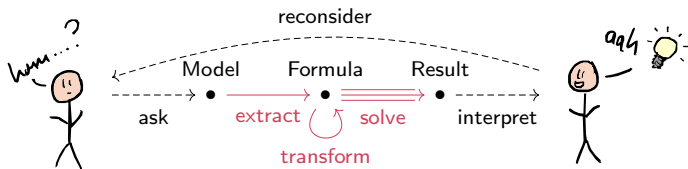
torte 🍰

Roll your own experiments!



 /ekuiter/ 🍰

torte – An Experimentation Platform For Feature-Model Analysis



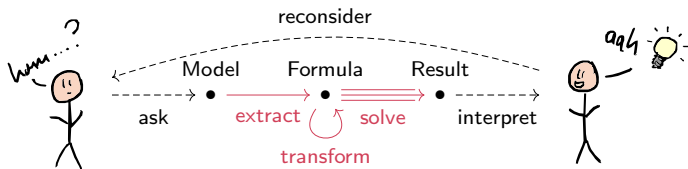
torte 🍰

Roll your own experiments!



 /ekuiter/ 🍰

torte – An Experimentation Platform For Feature-Model Analysis



torte 🍰

Roll your own experiments!



 /ekuiter/ 🍰

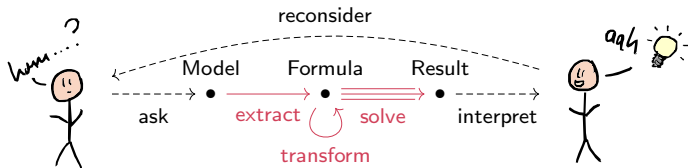
Extraction

KCONFIGREADER
KCLAUSE
CONFIGFIX
Hierarchy extractor
Benchmark models

Transformation

FEATUREIDE
FEATJAR
CLAUSY
Z3
CADI BACK

torte – An Experimentation Platform For Feature-Model Analysis



torte 🍰

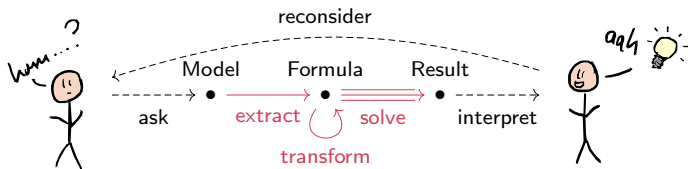
Roll your own experiments!



 /ekuitier/ 🍰

Extraction	Transformation	Solving
KCONFIGREADER	FEATUREIDE	53 SAT
KCLAUSE	FEATJAR	14 #SAT
CONFIGFIX	CLAUSY	SATGRAF
Hierarchy extractor	Z3	JUPYTER notebooks
Benchmark models	CADIBACK	Web dashboard

torte – An Experimentation Platform For Feature-Model Analysis



torte 🍰

Roll your own experiments!



 /ekuiter/ 🍰

Extraction	Transformation	Solving
KCONFIGREADER	FEATUREIDE	53 SAT
KCLAUSE	FEATJAR	14 #SAT
CONFIGFIX	CLAUSY	SATGRAF
Hierarchy extractor	Z3	JUPYTER notebooks
Benchmark models	CADIBACK	Web dashboard
Experimentation Platform		
Docker containerization	Job parallelization	Stage architecture
Continuous integration	Profiling system	Experiment DSL
Reproduction packages	Resource limiting	Remote execution

torte – Short Demonstration

torte – Short Demonstration

How configurable is BusyBox?



torte – Short Demonstration

How configurable is BusyBox?



Self-Executing One-Liner

```
curl -sL https://elias-kuiter.de/torte/ | sh
```

Requirements Docker, Git, curl, and GNU tools

torte – Short Demonstration

```
#!/bin/bash
TORTE_REVISION=7a65a22
TIMEOUT=10
JOBS=4
[[ $TOOL != torte ]] && builtin source /dev/stdin <<<"$(curl -fsSL \
https://raw.githubusercontent.com/ekuiter/torte/7a65a22/torte.sh)"

experiment-systems() {
    add-busybox-kconfig-history --from 1_36_0 --to 1_36_1
}

experiment-stages() {
    clone-systems; read-statistics; extract-kconfig-models

    # Serializes the feature model of BusyBox as UVL
    transform-model-with-featjar \
    --transformer transform-model-to-uvl-with-featureide \
    --output-extension uvl
    transform-model-to-dimacs

    # Counts the number of valid configurations of BusyBox
    solve-sharp-sat --timeout "$TIMEOUT" --jobs "$JOBS"
}
```

How configurable is BusyBox?



Self-Executing One-Liner

```
curl -sL https://elias-kuitert.de/torte/ | sh
```

Requirements Docker, Git, curl, and GNU tools

torte – Short Demonstration

```
#!/bin/bash
TORTE_REVISION=7a65a22
TIMEOUT=10
JOBS=4
[[ $TOOL != torte ]] && builtin source /dev/stdin <<<"$(curl -fsSL \
https://raw.githubusercontent.com/ekuiter/torte/7a65a22/torte.sh)"
```

```
experiment-systems() {
    add-busybox-kconfig-history --from 1_36_0 --to 1_36_1
}
```

```
experiment-stages() {
    clone-systems; read-statistics; extract-kconfig-models
```

```
# Serializes the feature model of BusyBox as UVL
transform-model-with-featjar \
--transformer transform-model-to-uvl-with-featureide \
--output-extension uvl
transform-model-to-dimacs
```

```
# Counts the number of valid configurations of BusyBox
solve-sharp-sat --timeout "$TIMEOUT" --jobs "$JOBS"
}
```

How configurable is BusyBox?

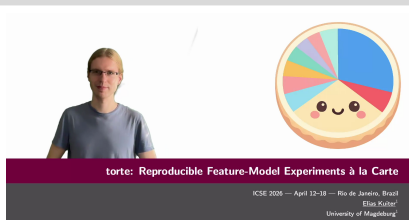


Self-Executing One-Liner

```
curl -sL https://elias-kuitert.de/torte/ | sh
```

Requirements Docker, Git, curl, and GNU tools

Demo Video



Experiences with torte

[illegible]

Experiences with torte

Tortin or not Tortin? The Impact of CNF Transformations on Feature-Model Analysers

Elias Kauter
Otto von Guericke University
Magdeburg, Germany
kauter@ovgu.de

Sébastien Krüger
University of Ulm
Ulmu, Germany
sebastien.krueger@uni-ulm.de

Chico Sundermann
University of Ulm
Ulmu, Germany
chico.sundermann@uni-ulm.de

Thomas Thüm
University of Ulm
Ulmu, Germany
thuem@uni-ulm.de

Günther Saake
Otto von Guericke University
Magdeburg, Germany
saake@ovgu.de

ABSTRACT

Feature modeling is widely used in systemically model-driven development of natural-rich software systems. Despite this, its impact on feature model analysis is largely unexplored. In this paper, we investigate the impact of CNF transformations on feature model analysis.

1. INTRODUCTION

Feature model analysis is widely used in systemically model-driven development of natural-rich software systems. Despite this, its impact on feature model analysis is largely unexplored. In this paper, we investigate the impact of CNF transformations on feature model analysis.

2. RELATED WORK

Feature model analysis is widely used in systemically model-driven development of natural-rich software systems. Despite this, its impact on feature model analysis is largely unexplored. In this paper, we investigate the impact of CNF transformations on feature model analysis.

3. CONCLUSIONS

Feature model analysis is widely used in systemically model-driven development of natural-rich software systems. Despite this, its impact on feature model analysis is largely unexplored. In this paper, we investigate the impact of CNF transformations on feature model analysis.

4. REFERENCES

Feature model analysis is widely used in systemically model-driven development of natural-rich software systems. Despite this, its impact on feature model analysis is largely unexplored. In this paper, we investigate the impact of CNF transformations on feature model analysis.

How Configurable is the Linux Kernel? Analyzing Two Decades of Feature-Model History

ELIAS KAUTER, University of Magdeburg, Magdeburg, Germany
CHICO SUNDERMANN and THOMAS THÜM, TU Braunschweig, Braunschweig, Germany
THOMAS HEID, University of Ulm, Ulm, Germany
SEBASTIAN KRÜGER, TU Braunschweig, Braunschweig, Germany
GÜNTHER SAAKE, University of Magdeburg, Magdeburg, Germany

Today, the operating system Linux is widely used in diverse environments, as its kernel can be configured flexibly. In many configurable systems, managing such variability can be facilitated in development phases with product-line analysis. These analyses often require knowledge about the system's features and their dependencies, which are documented in a feature model. Despite these potential, product-line analyses are rarely applied to the Linux kernel in practice, as the feature model still challenges scalability and accuracy of analysis. Unfortunately, these challenges also severely limit our knowledge about the fundamental attributes of the kernel's configurability, namely its number of features and configurations. We identify five key limitations in the literature related to the scalability, accuracy, and knowledge of these metrics, and, by extension, other product-line analyses (1). Analysis results for the Linux kernel are not comparable, because relevant information is not reported; (2) there is no consensus on how to define features in Linux, which leads to flawed analysis results; (3) only few versions of the Linux kernel have ever been analyzed, none of which are recent; and (4) the kernel is presented as complete, although we lack empirical evidence that supports this claim. In this article, we address these limitations with a comprehensive, empirical study of the Linux kernel's configurability, which spans its feature model's entire history from 2001 to 2018. We address the above limitations as follows: (1) We characterize parameters that are relevant when reporting analysis results; (2) we propose and evaluate a novel definition of features in Linux as a standardization effort; (3) we consolidate results, a task that analyzing arbitrary versions of the Linux kernel's feature model; and (4) we investigate the current and possible future configurability of the kernel in more than 1,000 feature model versions. Based on our results, we highlight a major insight into the Linux kernel's configurability and make it accessible for researchers and practitioners.

CCS CONCEPTS • Software and its engineering → Software product lines, Software evolution; Theory of computation → Automated reasoning.

Additional Key Words and Phrases: Linux kernel, feature modeling, software product lines, software variability

Authors' Contact Information: Elias Kauter (kauter@ovgu.de)

ASE'22

TOSEM'25

Investigating the Effects of T-Wise Interaction Sampling for Vulnerability Discovery in Highly-Configurable Software Systems

Tim Bockel
KAIST, Institute of Information Security and Dependability
Kardarobe, Germany
bockel@kardarobe.de

Elis Hahnke
KAIST, Institute of Information Security and Dependability
Kardarobe, Germany
hahnke@kardarobe.de

Christoph König
KAIST, Institute of Information Security and Dependability
Kardarobe, Germany
koenig@kardarobe.de

Tobias Pöt
KAIST, Institute of Information Security and Dependability
Kardarobe, Germany
poet@kardarobe.de

Isa Schuster
KAIST, Institute of Information Security and Dependability
Kardarobe, Germany
schuster@kardarobe.de

ABSTRACT

Feature model analysis has become a standard technique to manage variability in highly-configurable software systems. In this paper, we investigate the effects of T-wise interaction sampling for vulnerability discovery in highly-configurable software systems. We study the impact of T-wise interaction sampling on the number of vulnerabilities found and the time to find them.

1. INTRODUCTION

Feature model analysis has become a standard technique to manage variability in highly-configurable software systems. In this paper, we investigate the effects of T-wise interaction sampling for vulnerability discovery in highly-configurable software systems. We study the impact of T-wise interaction sampling on the number of vulnerabilities found and the time to find them.

2. RELATED WORK

Feature model analysis has become a standard technique to manage variability in highly-configurable software systems. In this paper, we investigate the effects of T-wise interaction sampling for vulnerability discovery in highly-configurable software systems. We study the impact of T-wise interaction sampling on the number of vulnerabilities found and the time to find them.

3. CONCLUSIONS

Feature model analysis has become a standard technique to manage variability in highly-configurable software systems. In this paper, we investigate the effects of T-wise interaction sampling for vulnerability discovery in highly-configurable software systems. We study the impact of T-wise interaction sampling on the number of vulnerabilities found and the time to find them.

Can SAT Solvers Keep Up with the Linux Kernel's Feature Model?

Elias Kauter
University of Magdeburg
Magdeburg, Germany

Udo Brandes-Rain
University of Magdeburg
Magdeburg, Germany

Thomas Thüm
University of Magdeburg
Magdeburg, Germany

Sébastien Krüger
University of Ulm
Ulmu, Germany

Günther Saake
University of Magdeburg
Magdeburg, Germany

Abstract

The Linux kernel is a highly relevant, yet also highly configurable software system. Even though it is widely used, its configurability is often overlooked. In this paper, we investigate the impact of SAT solvers on the analysis of the Linux kernel's feature model. We study the impact of SAT solvers on the number of vulnerabilities found and the time to find them.

1. INTRODUCTION

The Linux kernel is a highly relevant, yet also highly configurable software system. Even though it is widely used, its configurability is often overlooked. In this paper, we investigate the impact of SAT solvers on the analysis of the Linux kernel's feature model. We study the impact of SAT solvers on the number of vulnerabilities found and the time to find them.

2. RELATED WORK

The Linux kernel is a highly relevant, yet also highly configurable software system. Even though it is widely used, its configurability is often overlooked. In this paper, we investigate the impact of SAT solvers on the analysis of the Linux kernel's feature model. We study the impact of SAT solvers on the number of vulnerabilities found and the time to find them.

3. CONCLUSIONS

The Linux kernel is a highly relevant, yet also highly configurable software system. Even though it is widely used, its configurability is often overlooked. In this paper, we investigate the impact of SAT solvers on the analysis of the Linux kernel's feature model. We study the impact of SAT solvers on the number of vulnerabilities found and the time to find them.

- 7 Zenodo artifacts
- 3 ACM "Reusable" badges
- part of 1 best artifact award
- 6 student theses and projects



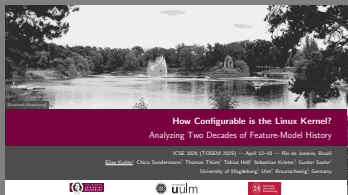
[illegible]

- 7 Zenodo artifacts
- 3 ACM “Reusable” badges
- part of 1 best artifact award
- 6 student theses and projects

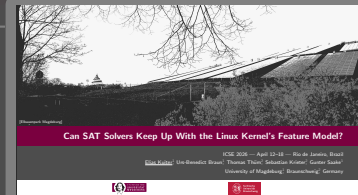


Conclusion

Journal-First Paper



Research Paper



How Configurable Is the Linux Kernel?

Can SAT Solvers Keep Up With the Linux Kernel's Feature Model?

Demo Paper



Image Credits

- **BusyBox logo**, BusyBox developers [\[https://commons.wikimedia.org/wiki/File:BusyBoxLogo.png\]](https://commons.wikimedia.org/wiki/File:BusyBoxLogo.png)
- **LEGO minifigures**, Thomas Thüm [\[https://github.com/TUBS-ISF/Course-on-Software-Product-Lines/tree/main/pics/lego\]](https://github.com/TUBS-ISF/Course-on-Software-Product-Lines/tree/main/pics/lego)
- **TikZpingus**, Florian Sihler [\[https://github.com/EagleoutIce/tikzpingus\]](https://github.com/EagleoutIce/tikzpingus)
- **Reusable Badge**, Association for Computing Machinery [\[https://www.acm.org/publications/policies/artifact-review-badging\]](https://www.acm.org/publications/policies/artifact-review-badging)

All images were used with permission. The slides were created with FancyBeamer and draw.io.